

AI ASSISTED CODING

Assignment Number: 13

E.Anjani

2303A52419

BATCH 32

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Lab Objectives:

- Identify code smells and inefficiencies in legacy Python scripts.
- Use AI-assisted coding tools to refactor for readability, maintainability, and performance.
- Apply modern Python best practices while ensuring output correctness.

Task Description #1 (Refactoring – Removing Code Duplication)

- Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.
- Instructions:
 - o Prompt AI to identify duplicate logic and replace it with functions or classes.
 - o Ensure the refactored code maintains the same output.
 - o Add docstrings to all functions.
- Sample Legacy Code:

```
# Legacy script with repeated logic  
print("Area of Rectangle:", 5 * 10)  
print("Perimeter of Rectangle:", 2 * (5 + 10))  
print("Area of Rectangle:", 7 * 12)  
print("Perimeter of Rectangle:", 2 * (7 + 12))  
print("Area of Rectangle:", 10 * 15)  
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

- Expected Output:
 - o Refactored code with a reusable function and no duplication.
 - o Well documented code

Code:

```

#identify the duplicate logic and correct the code and replace it with functions or classes
#and refactor the below code
#add docstrings to the code
def calculate_rectangle_area(length, width):
    """
    Calculate the area of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Returns:
        float: The area of the rectangle.
    """
    return length * width

def calculate_rectangle_perimeter(length, width):
    """
    Calculate the perimeter of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Returns:
        float: The perimeter of the rectangle.
    """
    return 2 * (length + width)

print("Area of Rectangle:", calculate_rectangle_area(5, 10))
print("Perimeter of Rectangle:", calculate_rectangle_perimeter(5, 10))
print("Area of Rectangle:", calculate_rectangle_area(7, 12))
print("Perimeter of Rectangle:", calculate_rectangle_perimeter(7, 12))
print("Area of Rectangle:", calculate_rectangle_area(10, 15))
print("Perimeter of Rectangle:", calculate_rectangle_perimeter(10, 15))

```

Output:

```

PS C:\Users\HP\OneDrive\Documents\AIAC_2026> & C:\Users\HP\OneDrive\Documents\AIAC_2026\Lab-11.py
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50

```

Task Description #2 (Refactoring – Extracting Reusable Functions)

- Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

- Instructions:

- o Identify repeated or related logic and extract it into reusable functions.

- o Ensure the refactored code is modular, easy to read, and documented with docstrings.

- Sample Legacy Code:

```
# Legacy script with inline repeated logic
```

```
price = 250
```

```
tax = price * 0.18
```

```
total = price + tax
```

```
print("Total Price:", total)
```

```
price = 500
```

```
tax = price * 0.18
```

```
total = price + tax
```

```
print("Total Price:", total)
```

- Expected Output:

- o Code with a function `calculate_total(price)` that can be reused for multiple price inputs.

- o Well documented code

Code:

```
#refactor the above code identify repeated or related logic and extract it into reusable function calculate_total(price) and
# generate a well documented code with docstrings
def calculate_total(price):
    """Calculate the total price including tax.

    Args:
        price (float): The original price of the item.

    Returns:
        float: The total price after adding tax.
    """
    tax = price * 0.18
    total = price + tax
    return total
# Example usage
if __name__ == "__main__":
    price1 = 250
    total1 = calculate_total(price1)
    print("Total Price:", total1)

    price2 = 500
    total2 = calculate_total(price2)
    print("Total Price:", total2)
```

Output:

```
Total Price: 295.0
Total Price: 590.0
Total Price: 295.0
Total Price: 590.0
```

Task Description #3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

Refactor a Python script that contains repeated if–elif–else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator

2. Method Name: calculate_grade(self, marks)

3. The method must:

- o Accept marks as a parameter.
- o Return the corresponding grade as a string.
- o The grading logic must strictly follow the conditions below:
 - $\text{Marks} \geq 90 \text{ and } \leq 100 \rightarrow \text{"Grade A"}$
 - $\text{Marks} \geq 80 \rightarrow \text{"Grade B"}$
 - $\text{Marks} \geq 70 \rightarrow \text{"Grade C"}$
 - $\text{Marks} \geq 40 \rightarrow \text{"Grade D"}$
 - $\text{Marks} \geq 0 \rightarrow \text{"Fail"}$

Note: Assume marks are within the valid range of 0 to 100.

4. Include proper docstrings for:

- o The class
- o The method (with parameter and return descriptions)

5. The method must be reusable and called multiple times without rewriting conditional logic.

• Given code:

```
marks = 85
```

```
if marks >= 90:
```

```
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
marks = 72
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
```

Expected Output:

- Define a class named GradeCalculator.
- Implement a method `calculate_grade(self, marks)` inside the class.
- Create an object of the class.
- Call the method for different student marks.
- Print the returned grade values.

Code:

```

#refactor the above code with a class-GradeCalculator and invoke a method calculate_grade(self,marks)
# to determine the grade based on the marks and generate a well documented code with docstrings
class GradeCalculator:
    def calculate_grade(self, marks):
        """Determine the grade based on the marks.

        Args:
            marks (int): The marks obtained by the student.

        Returns:
            str: The grade corresponding to the marks.
        """
        if marks >= 90 and marks <= 100:
            return "Grade A"
        elif marks >= 80:
            return "Grade B"
        elif marks >= 70:
            return "Grade C"
        elif marks >= 40:
            return "Grade D"
        elif marks >= 0:
            return "Grade F"
        else:
            return "Invalid marks"

# Example usage
if __name__ == "__main__":
    grade_calculator = GradeCalculator()

    marks1 = 85
    grade1 = grade_calculator.calculate_grade(marks1)
    print(f"Marks: {marks1}, {grade1}")

    marks2 = 72
    grade2 = grade_calculator.calculate_grade(marks2)
    print(f"Marks: {marks2}, {grade2}")

```

Output:

```

Marks: 85, Grade B
Marks: 72, Grade C
PS C:\Users\Admin\AppData\Local\Programs\Python\Python39\Scripts>

```

Task Description #4 (Refactoring – Converting Procedural Code to Functions)

- Task: Use AI to refactor procedural input–processing logic into functions.

Instructions:

- o Identify input, processing, and output sections.
- o Convert each into a separate function.
- o Improve code readability without changing behavior.

- Sample Legacy Code:

```
num = int(input("Enter number: "))
```

```
square = num * num
```

```
print("Square:", square)
```

- Expected Output:

- o Modular code using functions like `get_input()`, `calculate_square()`, and `display_result()`.

Code:

```
#refactor the above code using seperate function with docstrings
def calculate_square(num):
    """Calculate the square of a number.

    Args:
        num (int): The number to be squared.

    Returns:
        int: The square of the input number.
    """
    return num * num
# Example usage
if __name__ == "__main__":
    num = int(input("Enter number: "))
    square = calculate_square(num)
    print("Square:", square)
```

Output:

```
Enter number: 5
Square: 25
```

Task 5 (Refactoring Procedural Code into OOP Design)

- Task: Use AI to refactor procedural code into a class-based design.

Focus Areas:

- o Object-Oriented principles
- o Encapsulation

Legacy Code:

salary = 50000

```
tax = salary * 0.2
```

```
net = salary - tax
```

```
print(net)
```

Expected Outcome:

- o A class like EmployeeSalaryCalculator with methods and attributes.

Code:

```
#refactor the above code with a class-EmployeeSalaryCalculator
# with methods and attributes and invoke object-oriented principle
# Encapsulation
class EmployeeSalaryCalculator:
    def __init__(self, salary):
        self.salary = salary

    def calculate_tax(self):
        """Calculate the tax based on the salary."""
        return self.salary * 0.2

    def calculate_net_salary(self):
        """Calculate the net salary after deducting tax."""
        tax = self.calculate_tax()
        net_salary = self.salary - tax
        return net_salary

# Example usage
if __name__ == "__main__":
    employee_salary_calculator = EmployeeSalaryCalculator(50000)
    net_salary = employee_salary_calculator.calculate_net_salary()
    print("Net Salary:", net_salary)
```

Output:

```
Net Salary: 40000.0
```

Task 6 (Optimizing Search Logic)

- Task: Refactor inefficient linear searches using appropriate data structures.

- Focus Areas:

- o Time complexity

- o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
```

```
name = input("Enter username: ")
```

```
found = False
```

```
for u in users:
```

```
    if u == name:
```

```
        found = True
```

```
print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

- o Use of sets or dictionaries with complexity justification

Code:

```
#refactor the above code using sets or dictionaries with optimized time complexity
users = {"admin", "guest", "editor", "viewer"} # Using a set for O(1) average time complexity
name = input("Enter username: ")
if name in users:
    print("Access Granted")
else:
    print("Access Denied")
```

Output:

```
Enter username: john
Access Denied
```

Task 7 – Refactoring the Library Management System

Problem Statement

You are provided with a poorly structured Library Management script that:

- Contains repeated conditional logic
- Does not use reusable functions
- Lacks documentation
- Uses print-based procedural execution
- Does not follow modular programming principles

Your task is to refactor the code into a proper format

1. Create a module `library.py` with functions:

o `add_book(title, author, isbn)`

o `remove_book(isbn)`

o `search_book(isbn)`

2. Insert triple quotes under each function and let Copilot complete the docstrings.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format.

5. Open the file in a browser.

Given Code

```
# Library Management System (Unstructured Version)

# This code needs refactoring into a proper module with documentation.

library_db = {}

# Adding first book

title = "Python Basics"

author = "John Doe"

isbn = "101"

if isbn not in library_db:

    library_db[isbn] = {"title": title, "author": author}

    print("Book added successfully.")

else:

    print("Book already exists.")

# Adding second book (duplicate logic)

title = "AI Fundamentals"

author = "Jane Smith"

isbn = "102"

if isbn not in library_db:

    library_db[isbn] = {"title": title, "author": author}

    print("Book added successfully.")
```

```
else:
    print("Book already exists.")
    # Searching book (repeated logic structure)
    isbn = "101"
    if isbn in library_db:
        print("Book Found:", library_db[isbn])
    else:
        print("Book not found.")
    # Removing book (again repeated pattern)
    isbn = "101"
    if isbn in library_db:
        del library_db[isbn]
        print("Book removed successfully.")
    else:
        print("Book not found.")
    # Searching again
    isbn = "101"
    if isbn in library_db:
        print("Book Found:", library_db[isbn])
    else:
        print("Book not found.")
```

Code:

```
39 #refactor the above code with add_book(title,author,isbn),search_book(isbn) and remove_book(isbn) functions and generate a well documented code with docstrings
40 library_db = {}
41 def add_book(title, author, isbn):
42     """Add a book to the library database.
43
44     Args:
45         title (str): The title of the book.
46         author (str): The author of the book.
47         isbn (str): The ISBN number of the book.
48
49     Returns:
50         str: A message indicating whether the book was added successfully or if it already exists.
51     """
52     if isbn not in library_db:
53         library_db[isbn] = {"title": title, "author": author}
54         return "Book added successfully."
55     else:
56         return "Book already exists."
57 def search_book(isbn):
58     """Search for a book in the library database by its ISBN.
59
60     Args:
61         isbn (str): The ISBN number of the book to search for.
62
63     Returns:
64         dict or str: The book details if found, or a message indicating that the book was not found.
65     """
66     if isbn in library_db:
67         return library_db[isbn]
68     else:
69         return "Book not found."
70 def remove_book(isbn):
71     """Remove a book from the library database by its ISBN.
72
73     Args:
74         isbn (str): The ISBN number of the book to remove.
```

```
        return "Book not found."
def remove_book(isbn):
    """Remove a book from the library database by its ISBN.

    Args:
        isbn (str): The ISBN number of the book to remove.
    Returns:
        str: A message indicating whether the book was removed successfully or if it was not found.
    """
    if isbn in library_db:
        del library_db[isbn]
        return "Book removed successfully."
    else:
        return "Book not found."

# Example usage
if __name__ == "__main__":
    print(add_book("Python Basics", "John Doe", "101"))
    print(add_book("AI Fundamentals", "Jane Smith", "102"))
    print(search_book("101"))
    print(remove_book("101"))
    print(search_book("101"))
```

Output:


```

PS C:\Users\Admin\Downloads\AI ASSISTENCE> & C:\Users\Admin\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/Admin/Downloads/AI ASSISTENCE/library.py"
Book added successfully.
Book added successfully.
{'title': 'Python Basics', 'author': 'John Doe'}
Book removed successfully.
Book not found.
PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc library
Help on module library:

NAME
    library - #refactor the above code with add_book(title,author,isbn),search_book(isbn) and remove_book(isbn) functions and generate a well documented code with docstrings

FUNCTIONS
    add_book(title, author, isbn)
        Add a book to the library database.

        Args:
            title (str): The title of the book.
            author (str): The author of the book.
            isbn (str): The ISBN number of the book.

        Returns:
            str: A message indicating whether the book was added successfully or if it already exists.

    remove_book(isbn)
        Remove a book from the library database by its ISBN.

        Args:
            isbn (str): The ISBN number of the book to remove.

        Returns:
            str: A message indicating whether the book was removed successfully or if it was not found.

    search_book(isbn)
        Search for a book in the library database by its ISBN.

        Args:
            isbn (str): The ISBN number of the book to search for.

```

```

            isbn (str): The ISBN number of the book.

        Returns:
            str: A message indicating whether the book was added successfully or if it already exists.

    remove_book(isbn)
        Remove a book from the library database by its ISBN.

        Args:
            isbn (str): The ISBN number of the book to remove.

        Returns:
            str: A message indicating whether the book was removed successfully or if it was not found.

    search_book(isbn)
        Search for a book in the library database by its ISBN.

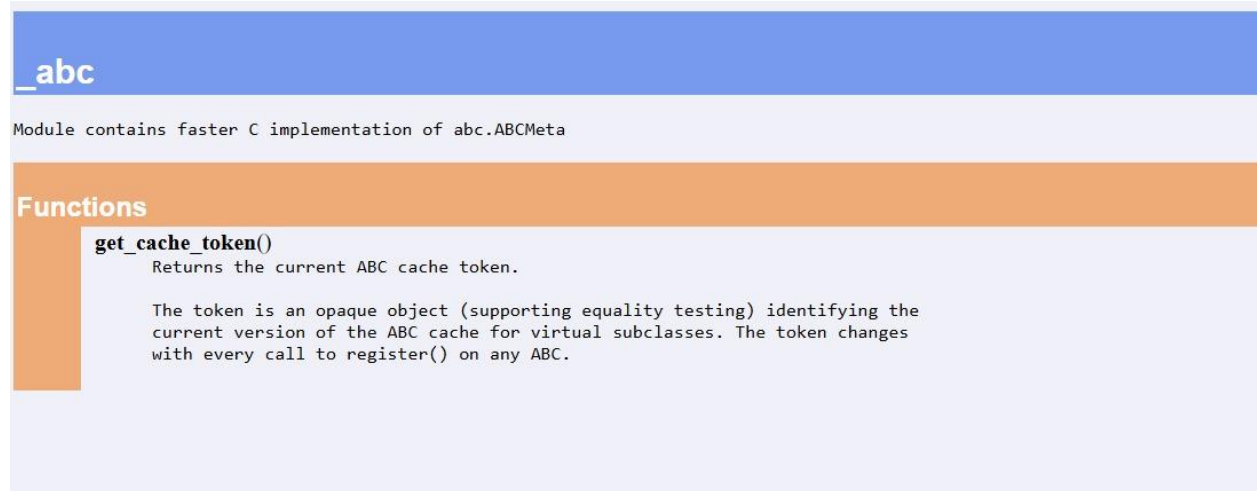
        Args:
            isbn (str): The ISBN number of the book to search for.

        Returns:
            dict or str: The book details if found, or a message indicating that the book was not found.

DATA
    library_db = {}

FILE
    c:\users\admin\downloads\ai assistance\library.py
-- More --

```



Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

- Global variables.
- Inefficient loop.
- No functions or modularity.

Task for Students:

- Refactor into a clean reusable function (generate_fibonacci).
- Add docstrings and test cases.

- Compare AI-refactored vs original.

Bad Code Version:

```
# fibonacci bad version
```

```
n=int(input("Enter limit: "))
```

```
a=0
```

```
b=1
```

```
print(a)
```

```
print(b)
```

```
for i in range(2,n):
```

```
    c=a+b
```

```
    print(c)
```

```
    a=b
```

```
    b=c
```

Code:

```
#refactor the above code using functio-generate_fibonacci with docstrings and generate a well documented code
def generate_fibonacci(n):
    """Generate Fibonacci sequence up to n terms.

    Args:
        n (int): The number of terms in the Fibonacci sequence to generate.

    Returns:
        list: A list containing the Fibonacci sequence up to n terms.
    """
    sequence = []
    a, b = 0, 1
    for i in range(n):
        sequence.append(a)
        a, b = b, a + b
    return sequence

# Example usage
if __name__ == "__main__":
    n = int(input("Enter limit: "))
    fibonacci_sequence = generate_fibonacci(n)
    print("Fibonacci Sequence:", fibonacci_sequence)
```

Output:

```
Enter limit: 5
Fibonacci Sequence: [0, 1, 1, 2, 3]
```

Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

- Inefficient prime checking.
- No functions.
- Hardcoded inputs.

Task for Students:

- Refactor into `is_prime(n)` and `is_twin_prime(p1, p2)`.
- Add docstrings and optimize.
- Generate a list of twin primes in a given range using AI.

Bad Code Version:

```
# twin primes bad version
```

```
a=11
```

```
b=13
```

```
fa=0
```

```
for i in range(2,a):
```

```
    if a%i==0:
```

```
        fa=1
```

```
fb=0
```

```
for i in range(2,b):
```

```
    if b%i==0:
```

```
        fb=1
```

```
if fa==0 and fb==0 and abs(a-b)==2:
```

```
    print("Twin Primes")
```

```
else:
```

```
    print("Not Twin Primes")
```

Code:

```
#refactor the above code using into is_prime(n) and is_twin_prime(p1,p2) with docstrings and generate a well documented code
def is_prime(n):
    """Check if a number is prime.

    Args:
        n (int): The number to check for primality.
    Returns:
        bool: True if the number is prime, False otherwise.

    """
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
#example usage
if __name__ == "__main__":
    a = 11
    b = 13

    if is_prime(a) and is_prime(b) and abs(a - b) == 2:
        print("Twin Primes")
    else:
        print("Not Twin Primes")
```

Output:

```
PS C:\Users\Admin\AppData\Local\Programs\Microsoft VS Code> 15
15
```

Task 10 – Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and

reusable implementation.

The current program reads a year from the user and prints the corresponding

Chinese Zodiac sign. However, the implementation contains repetitive conditional logic, lacks modular design, and does not follow clean coding

principles.

Your task is to refactor the code to improve readability, maintainability, and structure.

Chinese Zodiac Cycle (Repeats Every 12 Years)

1. Rat
2. Ox
3. Tiger
4. Rabbit
5. Dragon
6. Snake
7. Horse
8. Goat (Sheep)
9. Monkey
10. Rooster
11. Dog
12. Pig

Chinese Zodiac Program (Unstructured Version)

This code needs refactoring.

```
year = int(input("Enter a year: "))
```

```
if year % 12 == 0:
```

```
    print("Monkey")
```

```
elif year % 12 == 1:
    print("Rooster")
elif year % 12 == 2:
    print("Dog")
elif year % 12 == 3:
    print("Pig")
elif year % 12 == 4:
    print("Rat")
elif year % 12 == 5:
    print("Ox")
elif year % 12 == 6:
    print("Tiger")
elif year % 12 == 7:
    print("Rabbit")
elif year % 12 == 8:
    print("Dragon")
elif year % 12 == 9:
    print("Snake")
elif year % 12 == 10:
    print("Horse")
elif year % 12 == 11:
```



```
print("Goat")
```

You must:

1. Create a reusable function: `get_zodiac(year)`
2. Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).
3. Add proper docstrings.
4. Separate input handling from logic.
5. Improve readability and maintainability.
6. Ensure output remains correct.

Code:

```
#refactor the above code with function-get_zodiac(year) with docstrings and generate a well documented code
# and replace the if-elif chain with a list or dictionary for better readability and maintainability
def get_zodiac(year):
    """Get the Chinese zodiac sign for a given year.

    Args:
        year (int): The year for which to determine the zodiac sign.
    Returns:
        str: The Chinese zodiac sign corresponding to the given year.
    """
    zodiac_signs = [
        "Monkey", "Rooster", "Dog", "Pig", "Rat", "Ox",
        "Tiger", "Rabbit", "Dragon", "Snake", "Horse", "Goat"
    ]
    return zodiac_signs[year % 12]
# Example usage
if __name__ == "__main__":
    year = int(input("Enter a year: "))
    zodiac_sign = get_zodiac(year)
    print(f"The Chinese zodiac sign for the year {year} is: {zodiac_sign}")
```

```
Enter a year: 2008
The Chinese zodiac sign for the year 2008 is: Rat
```

Task 11 – Refactoring the Harshad (Niven) Number Checker

Refactor the given poorly structured Python script into a clean, modular, and

reusable implementation.

A Harshad (Niven) number is a number that is divisible by the sum of its digits.

For example:

- $18 \rightarrow 1 + 8 = 9 \rightarrow 18 \div 9 = 2$ (Harshad Number)
- $19 \rightarrow 1 + 9 = 10 \rightarrow 19 \div 10 \neq \text{integer}$ (Not Harshad)

Problem Statement

The current implementation:

- Mixes logic and input handling
- Uses redundant variables
- Does not use reusable functions properly
- Returns print statements instead of boolean values
- Lacks documentation

You must refactor the code to follow clean coding principles.

Harshad Number Checker (Unstructured Version)

```
num = int(input("Enter a number: "))
```

```
temp = num
```

```
sum_digits = 0
```

```
while temp > 0:
```

```
digit = temp % 10
sum_digits = sum_digits + digit
temp = temp // 10
if sum_digits != 0:
    if num % sum_digits == 0:
        print("True")
    else:
        print("False")
else:
    print("False")
```

You must:

1. Create a reusable function: `is_harshad(number)`
2. The function must:
 - o Accept an integer parameter.
 - o Return True if the number is divisible by the sum of its digits.
 - o Return False otherwise.
3. Separate user input from core logic.
4. Add proper docstrings.
5. Improve readability and maintainability.
6. Ensure the program handles edge cases (e.g., 0, negative numbers).

```

#refactor the above code using function-is_harshad(number) and this function should accept an integer and
# return boolean and handle edge cases with docstrings and generate a well documented code
def is_harshad(number):
    """Check if a number is a Harshad number.

    A Harshad number is an integer that is divisible by the sum of its digits.

    Args:
        number (int): The number to check.
    Returns:
        bool: True if the number is a Harshad number, False otherwise.
    """
    if number < 0:
        return False # Harshad numbers are typically defined for non-negative integers

    temp = number
    sum_digits = 0

    while temp > 0:
        digit = temp % 10
        sum_digits += digit
        temp //= 10

    if sum_digits == 0:
        return False # Avoid division by zero

    return number % sum_digits == 0
# Example usage
if __name__ == "__main__":
    num = int(input("Enter a number: "))
    if is_harshad(num):
        print("True")
    else:
        print("False")

```

```

Enter a number: 22
False

```

Task 12 – Refactoring the Factorial Trailing Zeros Program

Refactor the given poorly structured Python script into a clean, modular, and

efficient implementation.

The program calculates the number of trailing zeros in $n!$ (factorial of n).

Problem Statement

The current implementation:

- Calculates the full factorial (inefficient for large n)
- Mixes input handling with business logic
- Uses print statements instead of return values
- Lacks modular structure and documentation

You must refactor the code to improve efficiency, readability, and maintainability.

Factorial Trailing Zeros (Unstructured Version)

```
n = int(input("Enter a number: "))
```

```
fact = 1
```

```
i = 1
```

```
while i <= n:
```

```
    fact = fact * i
```

```
    i = i + 1
```

```
count = 0
```

```
while fact % 10 == 0:
```

```
    count = count + 1
```

```
    fact = fact // 10
```

```
print("Trailing zeros:", count)
```

You must:

1. Create a reusable function: `count_trailing_zeros(n)`

2. The function must:
 - o Accept a non-negative integer n.
 - o Return the number of trailing zeros in n!.
3. Do NOT compute the full factorial.
4. Use an optimized mathematical approach (count multiples of 5).
5. Add proper docstrings.
6. Separate user interaction from core logic.
7. Handle edge cases (e.g., negative numbers, zero).

```
#refactor the above code using function-count_trailing_zeros(n) with docstrings and
# generate a well documented code and handle edge cases the function must be accept a non-negative integer
# n and return the number of trailing zeroes in n!
def count_trailing_zeros(n):
    """Count the number of trailing zeros in n!.

    Trailing zeros in n! are caused by the factors of 10, which are made of 2 and 5.
    Since there are usually more factors of 2 than 5 in a factorial, we can count the number of factors of 5 to determine the number
    of trailing zeros.

    Args:
        n (int): The non-negative integer for which to count trailing zeros in its factorial.
    Returns:
        int: The number of trailing zeros in n!.
    """
    if n < 0:
        raise ValueError("Input must be a non-negative integer.")

    count = 0
    power_of_5 = 5

    while power_of_5 <= n:
        count += n // power_of_5
        power_of_5 *= 5

    return count
# Example usage
if __name__ == "__main__":
    n = int(input("Enter a number: "))
    try:
        trailing_zeros = count_trailing_zeros(n)
        print("Trailing zeros in {}!: {}".format(n, trailing_zeros))
    except ValueError as e:
        print(e)
```

```
Enter a number: 22
Trailing zeros in 22!: 4
```

Task Description #13: Smart Airport Management System – Data Structure

Challenge

An airport authority plans to implement:

1. Passenger Check-In Queue – Process passengers in order of arrival.
2. Emergency Landing Priority – Aircraft in emergency get priority.
3. Flight Information Lookup – Fast search using Flight ID.
4. Air Route Connectivity – Airports connected via routes.
5. Boarding Process (Last-In First-Out Cabin Storage) – Overhead luggage

management simulation.

Student Task:

- Choose suitable data structures from the provided list.
- Provide 2–3 sentence justification per feature.
- Implement one feature in Python with comments and docstrings using AI assistance.

Expected Output:

- Mapping table.
- Working Python implementation.

```

#generate a python code for priority queue(emergency landing priority) and graphs(air rooute connectivity)with examples
import heapq
class EmergencyLandingPriorityQueue:
    def __init__(self):
        self.queue = []

    def add_flight(self, flight_number, priority):
        """Add a flight to the priority queue.

        Args:
            flight_number (str): The flight number.
            priority (int): The priority level (lower number means higher priority).
        """
        heapq.heappush(self.queue, (priority, flight_number))

    def get_next_flight(self):
        """Get the next flight for emergency landing based on priority.

        Returns:
            str: The flight number of the next flight for emergency landing.
        """
        if not self.queue:
            return None
        return heapq.heappop(self.queue)[1]

# Example usage
if __name__ == "__main__":
    landing_queue = EmergencyLandingPriorityQueue()
    landing_queue.add_flight("Flight A", 2)
    landing_queue.add_flight("Flight B", 1)
    landing_queue.add_flight("Flight C", 3)

    print("Next flight for emergency landing:", landing_queue.get_next_flight()) # Should return "Flight B"
    print("Next flight for emergency landing:", landing_queue.get_next_flight()) # Should return "Flight A"
    print("Next flight for emergency landing:", landing_queue.get_next_flight()) # Should return "Flight C"

```



```

import networkx as nx
import matplotlib.pyplot as plt
class AirRouteGraph:
    def __init__(self):
        self.graph = nx.Graph()

    def add_route(self, city1, city2):
        """Add a route between two cities.

        Args:
            city1 (str): The first city.
            city2 (str): The second city.
        """
        self.graph.add_edge(city1, city2)

    def display_routes(self):
        """Display the air route connectivity graph."""
        nx.draw(self.graph, with_labels=True, node_color='lightblue', node_size=500)
        plt.show()

# Example usage
if __name__ == "__main__":
    air_route_graph = AirRouteGraph()
    air_route_graph.add_route("City A", "City B")
    air_route_graph.add_route("City B", "City C")
    air_route_graph.add_route("City A", "City C")
    air_route_graph.add_route("City C", "City D")

    print("Air route connectivity graph:")
    air_route_graph.display_routes()

```

```

Next flight for emergency landing: Flight B
Next flight for emergency landing: Flight A
Next flight for emergency landing: Flight C

```

Feature	Data Structure	Justification
Passenger Check-In Queue	Queue	Passengers must be processed in the exact order they arrive. A queue follows FIFO (First In First Out), which ensures fairness and prevents skipping.
Emergency Landing Priority	Priority Queue	Emergency aircraft must be handled before normal flights. A priority queue always removes the highest-priority element first, making it ideal for emergency handling.
Flight	Hash Table	Flight IDs must be searched instantly. A hash table provides

Feature	Data Structure	Justification
Information Lookup		average $O(1)$ lookup time, making retrieval fast and efficient.
Air Route Connectivity	Graph	Airports and routes form a network where each airport connects to others. Graphs represent nodes (airports) and edges (routes), allowing path and connectivity analysis.
Boarding Cabin Storage	Stack	Cabin luggage is stored in overhead bins where the last placed bag is often removed first. Stack follows LIFO (Last In First Out), matching this behavior.

Task Description #14: Smart Social Media Platform – Data Structure Selection

A social media company wants to develop a system that includes:

1. News Feed Generation – Display posts chronologically.
2. Trending Hashtags Tracker – Rank hashtags by usage frequency.
3. User Profile Search – Quick lookup using username.
4. Friend Network Representation – Represent user connections.
5. Message Undo Feature – Allow last sent message to be withdrawn.

Student Task:

- Select appropriate data structures from the list.
- Justify your choices clearly.
- Implement one selected feature as a Python program using AI-assisted code

generation.

Expected Output:

- Table mapping features to data structures with justification.
- Working Python code with docstrings and comments.

```
#generate a python code for stack(message undo feature) with proper documentation ,docstrings and example
class MessageStack:
    def __init__(self):
        self.stack = []

    def send_message(self, message):
        """Send a message and add it to the stack.

        Args:
            message (str): The message to be sent.
        """
        self.stack.append(message)
        print(f"Message sent: {message}")

    def undo_message(self):
        """Undo the last sent message.

        Returns:
            str: The message that was undone, or None if there are no messages to undo.
        """
        if not self.stack:
            print("No messages to undo.")
            return None
        undone_message = self.stack.pop()
        print(f"Message undone: {undone_message}")
        return undone_message

# Example usage
if __name__ == "__main__":
    message_stack = MessageStack()
    message_stack.send_message("Hello, World!")
    message_stack.send_message("How are you?")
    message_stack.undo_message() # Should undo "How are you?"
    message_stack.undo_message() # Should undo "Hello, World!"
    message_stack.undo_message() # Should indicate no messages to undo
```

```
Message sent: Hello, World!
Message sent: How are you?
Message undone: How are you?
Message undone: Hello, World!
No messages to undo.
```

Feature	Data Structure	Justification
---------	----------------	---------------

Feature	Data Structure	Justification
News Feed Generation	Linked List	Posts are constantly added and removed. Linked lists allow efficient insertion and deletion without shifting elements.
Trending Hashtags Tracker	Priority Queue	Hashtags must be ranked by popularity. A priority queue keeps the highest frequency hashtags at the top automatically.
User Profile Search	Hash Table	Username must be found instantly. Hash tables allow fast $O(1)$ average lookup using username as key.
Friend Network Representation	Graph	Users connect with many other users. A graph structure models nodes (users) and edges (friendships).
Message Undo Feature	Stack	The most recent message should be undone first. Stack's LIFO behavior perfectly supports undo operations.

Task #15: Smart University Hostel Management

Scenario:

A university wants to build a hostel management system that manages:

1. Room Allotment
2. Medical Emergency Handling
3. Student Record Retrieval
4. Hostel Block Connectivity
5. Visitor Entry Log

Student Task

- For each feature, select the most appropriate data structure from the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```
#generate a python code for room allotment using queue with proper documentation ,docstrings and example
from collections import deque
class RoomAllotmentQueue:
    def __init__(self):
        self.queue = deque()

    def add_guest(self, guest_name):
        """Add a guest to the room allotment queue.

        Args:
            guest_name (str): The name of the guest to be added.
        """
        self.queue.append(guest_name)
        print(f"Guest added to queue: {guest_name}")

    def allot_room(self):
        """Allot a room to the next guest in the queue.

        Returns:
            str: The name of the guest who was allotted a room, or None if there are no guests in the queue.
        """
        if not self.queue:
            print("No guests in the queue to allot a room.")
            return None
        allotted_guest = self.queue.popleft()
        print(f"Room allotted to: {allotted_guest}")
        return allotted_guest

# Example usage
if __name__ == "__main__":
    room_allotment_queue = RoomAllotmentQueue()
    room_allotment_queue.add_guest("Alice")
    room_allotment_queue.add_guest("Bob")
    room_allotment_queue.add_guest("Charlie")

    room_allotment_queue.allot_room() # Should allot a room to Alice
    room_allotment_queue.allot_room() # Should allot a room to Bob
    room_allotment_queue.allot_room() # Should allot a room to Charlie
```

```
Guest added to queue: Alice
Guest added to queue: Bob
Guest added to queue: Charlie
Room allotted to: Alice
Room allotted to: Bob
Room allotted to: Charlie
No guests in the queue to allot a room.
```

Feature	Data Structure	Justification
Room Allotment	Queue	Students should receive rooms based on application order. Queue ensures fair FIFO allocation.
Medical Emergency Handling	Priority Queue	Severe medical cases must be handled before minor ones. Priority queue processes highest urgency first.

Feature	Data Structure	Justification
Student Record Retrieval	Hash Table	Student records must be accessed quickly using ID. Hash tables provide constant time lookup.
Hostel Block Connectivity	Graph	Hostel blocks and pathways form a connected network. Graph structures represent this connectivity clearly.
Visitor Entry Log	Stack	The most recent visitor entry may need quick removal or verification. Stack allows easy last-entry access.

Task #16: Smart Electricity Distribution System

Scenario:

An electricity board plans to develop a system that includes:

1. Complaint Handling
2. Fault Priority Handling
3. Consumer Record Lookup
4. Power Grid Connectivity
5. Meter Reading History Tracking

Student Task

- For each feature, select the most appropriate data structure from the list

below:

- o Stack
- o Queue
- o Priority Queue

- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.


```
#generate a python code for complaint handling using queue with proper documentation ,docstrings and example
from collections import deque
class ComplaintHandlingQueue:
    def __init__(self):
        self.queue = deque()

    def add_complaint(self, complaint):
        """Add a complaint to the handling queue.

        Args:
            complaint (str): The complaint to be added.
        """
        self.queue.append(complaint)
        print(f"Complaint added: {complaint}")

    def handle_complaint(self):
        """Handle the next complaint in the queue.

        Returns:
            str: The complaint that was handled, or None if there are no complaints to handle.
        """
        if not self.queue:
            print("No complaints to handle.")
            return None
        handled_complaint = self.queue.popleft()
        print(f"Complaint handled: {handled_complaint}")
        return handled_complaint

# Example usage
if __name__ == "__main__":
    complaint_queue = ComplaintHandlingQueue()
    complaint_queue.add_complaint("Late delivery")
    complaint_queue.add_complaint("Damaged product")
    complaint_queue.add_complaint("Wrong item received")

    complaint_queue.handle_complaint() # Should handle "Late delivery"
    complaint_queue.handle_complaint() # Should handle "Damaged product"
    complaint_queue.handle_complaint() # Should handle "Wrong item received"
```

```
Complaint added: Late delivery
Complaint added: Damaged product
Complaint added: Wrong item received
Complaint handled: Late delivery
Complaint handled: Damaged product
Complaint handled: Wrong item received
No complaints to handle.
```

Feature	Data Structure	Justification
Complaint Handling	Queue	Complaints should be resolved in order of submission. Queue maintains fairness using FIFO processing.
Fault Priority Handling	Priority Queue	Critical faults such as power outages must be fixed before minor issues. Priority queue handles urgent faults first.

Feature	Data Structure	Justification
Consumer Record Lookup	Hash Table	Consumer data must be retrieved quickly using ID or meter number. Hash tables enable constant time search.
Power Grid Connectivity	Graph	Power stations and transmission lines form a network. Graphs represent nodes and connections efficiently.
Meter Reading History	Linked List	Meter readings are continuously appended over time. Linked lists allow efficient insertion without resizing.

Task #17: Smart Logistics & Warehouse Management

Scenario:

A logistics company wants to implement:

1. Shipment Processing
2. Urgent Shipment Priority
3. Product Lookup by Code
4. Warehouse Network Connectivity
5. Inventory Add/Remove Operations

Student Task

- For each feature, select the most appropriate data structure from the list

below:

- o Stack
- o Queue
- o Priority Queue

- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Test Cases Design

Task 18 (Collatz Sequence Generator – Test Case Design)

- Function: Generate Collatz sequence until reaching 1.
- Test Cases to Design:
- Normal: $6 \rightarrow [6, 3, 10, 5, 16, 8, 4, 2, 1]$
- Edge: $1 \rightarrow [1]$
- Negative: -5

- Large: 27 (well-known long sequence)
- Requirement: Validate correctness with pytest.

Explanation:

We need to write a function that:

- Takes an integer n as input.
- Generates the Collatz sequence (also called the $3n+1$ sequence).
- The rules are:
 - If n is even $\rightarrow \text{next} = n / 2$.
 - If n is odd $\rightarrow \text{next} = 3n + 1$.
- Repeat until we reach 1.
- Return the full sequence as a list.

Example

Input: 6

Steps:

- 6 (even $\rightarrow 6/2 = 3$)
- 3 (odd $\rightarrow 3*3+1 = 10$)
- 10 (even $\rightarrow 10/2 = 5$)
- 5 (odd $\rightarrow 3*5+1 = 16$)
- 16 (even $\rightarrow 16/2 = 8$)
- 8 (even $\rightarrow 8/2 = 4$)
- 4 (even $\rightarrow 4/2 = 2$)

- 2 (even $\rightarrow 2/2 = 1$)

Output:

[6, 3, 10, 5, 16, 8, 4, 2, 1]

Task 19 (Lucas Number Sequence – Test Case Design)

- Function: Generate Lucas sequence up to n terms.

(Starts with 2,1, then $F_n = F_{n-1} + F_{n-2}$)

- Test Cases to Design:
- Normal: 5 $\rightarrow$ [2, 1, 3, 4, 7]
- Edge: 1 $\rightarrow$ [2]
- Negative: -5 $\rightarrow$ Error
- Large: 10 (last element = 76).
- Requirement: Validate correctness with pytest.

Code:

```

#generate a python code for queue for shipment processing with proper documentation ,docstrings and example
from collections import deque
class ShipmentProcessingQueue:
    def __init__(self):
        self.queue = deque()

    def add_shipment(self, shipment_id):
        """Add a shipment to the processing queue.

        Args:
            shipment_id (str): The ID of the shipment to be added.
        """
        self.queue.append(shipment_id)
        print(f"Shipment added: {shipment_id}")

    def process_shipment(self):
        """Process the next shipment in the queue.

        Returns:
            str: The ID of the shipment that was processed, or None if there are no shipments to process.
        """
        if not self.queue:
            print("No shipments to process.")
            return None
        processed_shipment = self.queue.popleft()
        print(f"Shipment processed: {processed_shipment}")
        return processed_shipment

```

```

# Example usage
if __name__ == "__main__":
    shipment_queue = ShipmentProcessingQueue()
    shipment_queue.add_shipment("Shipment001")
    shipment_queue.add_shipment("Shipment002")
    shipment_queue.add_shipment("Shipment003")

    shipment_queue.process_shipment() # Should process "Shipment001"
    shipment_queue.process_shipment() # Should process "Shipment002"
    shipment_queue.process_shipment() # Should process "Shipment003"
    shipment_queue.process_shipment() # Should indicate no shipments to process

```

```

Shipment added: Shipment001
Shipment added: Shipment002
Shipment added: Shipment003
Shipment processed: Shipment001
Shipment processed: Shipment002
Shipment processed: Shipment003
No shipments to process.

```

Feature	Data Structure	Justification
Shipment Processing	Queue	Shipments should be handled in the order received. Queue ensures FIFO processing.
Urgent Shipment Priority	Priority Queue	High priority shipments must be delivered earlier than normal ones. Priority queue sorts shipments by urgency.
Product Lookup by Code	Hash Table	Product codes require fast retrieval. Hash tables allow constant time lookup.
Warehouse Network Connectivity	Graph	Warehouses and transport routes form a network. Graph models connections and route planning.
Inventory Add/Remove	Stack	Recently stored items may be removed first during packing or restocking. Stack supports LIFO operations.

Task 18 (Collatz Sequence Generator – Test Case Design)

- Function: Generate Collatz sequence until reaching 1.
- Test Cases to Design:
- Normal: $6 \rightarrow [6, 3, 10, 5, 16, 8, 4, 2, 1]$
- Edge: $1 \rightarrow [1]$
- Negative: -5
- Large: 27 (well-known long sequence)
- Requirement: Validate correctness with pytest.

Explanation:

We need to write a function that:

- Takes an integer n as input.

- Generates the Collatz sequence (also called the $3n+1$ sequence).
- The rules are:
 - If n is even $\rightarrow \text{next} = n / 2$.
 - If n is odd $\rightarrow \text{next} = 3n + 1$.
- Repeat until we reach 1.
- Return the full sequence as a list.

Example

Input: 6

Steps:

- 6 (even $\rightarrow 6/2 = 3$)
- 3 (odd $\rightarrow 3*3+1 = 10$)
- 10 (even $\rightarrow 10/2 = 5$)
- 5 (odd $\rightarrow 3*5+1 = 16$)
- 16 (even $\rightarrow 16/2 = 8$)
- 8 (even $\rightarrow 8/2 = 4$)
- 4 (even $\rightarrow 4/2 = 2$)
- 2 (even $\rightarrow 2/2 = 1$)

Output:

[6, 3, 10, 5, 16, 8, 4, 2, 1]


```
#generate a python code with a function to calculate the Collatz sequence for a given integer
# with proper documentation and docstrings
def collatz_sequence(n):
    """Generate the Collatz sequence for a given integer n.

    Args:
        n (int): The starting integer for the Collatz sequence.
    Returns:
        list: A list containing the Collatz sequence starting from n and ending at 1.
    """
    if n <= 0:
        raise ValueError("Input must be a positive integer.")

    sequence = []
    while n != 1:
        sequence.append(n)
        if n % 2 == 0:
            n = n // 2
        else:
            n = 3 * n + 1
    sequence.append(1) # Append the final element of the sequence
    return sequence
```

```
# Example usage
if __name__ == "__main__":
    print(collatz_sequence(6)) # Output: [6, 3, 10, 5, 16, 8, 4, 2, 1]
    print(collatz_sequence(1)) # Output: [1]
    try:
        print(collatz_sequence(-5)) # Should raise ValueError
    except ValueError as e:
        print(e)
    print(collatz_sequence(27)) # Output: A long sequence ending in 1
```

```
[6, 3, 10, 5, 16, 8, 4, 2, 1]
[1]
Input must be a positive integer.
[27, 82, 41, 124, 62, 31, 94, 47, 142, 71, 214, 107, 322, 161, 484, 242, 121, 364, 182, 91, 274, 137, 412, 206, 103, 310, 155, 466, 233, 700, 350, 175, 526,
263, 790, 395, 1186, 593, 1780, 890, 445, 1336, 668, 334, 167, 502, 251, 754, 377, 1132, 566, 283, 850, 425, 1276, 638, 319, 958, 479, 1438, 719, 2158, 1079,
3238, 1619, 4858, 2429, 7288, 3644, 1822, 911, 2734, 1367, 4102, 2051, 6154, 3077, 9232, 4616, 2308, 1154, 577, 1732, 866, 433, 1300, 650, 325, 976, 488, 24
4, 122, 61, 184, 92, 46, 23, 70, 35, 106, 53, 160, 80, 40, 20, 10, 5, 16, 8, 4, 2, 1]
```

Task 19 (Lucas Number Sequence – Test Case Design)

- Function: Generate Lucas sequence up to n terms.

(Starts with 2,1, then $F_n = F_{n-1} + F_{n-2}$)

- Test Cases to Design:

- Normal: $5 \rightarrow [2, 1, 3, 4, 7]$
- Edge: $1 \rightarrow [2]$
- Negative: $-5 \rightarrow \text{Error}$
- Large: 10 (last element = 76).
- Requirement: Validate correctness with pytest.

```
#generate a python code with a function to calculate the Lucas sequence up to n terms with proper documentation and docstrings
def lucas_sequence(n):
    """Generate the Lucas sequence up to n terms.

    The Lucas sequence starts with 2 and 1, and each subsequent term is the sum of the two preceding terms.

    Args:
        n (int): The number of terms in the Lucas sequence to generate.
    Returns:
        list: A list containing the Lucas sequence up to n terms.
    """
    if n < 1:
        raise ValueError("Input must be a positive integer.")

    sequence = []
    a, b = 2, 1
    for i in range(n):
        sequence.append(a)
        a, b = b, a + b
    return sequence

# Example usage
if __name__ == "__main__":
    print(lucas_sequence(5)) # Output: [2, 1, 3, 4, 7]
    print(lucas_sequence(1)) # Output: [2]
    try:
        print(lucas_sequence(-5)) # Should raise ValueError
    except ValueError as e:
        print(e)
    print(lucas_sequence(10)) # Output: [2, 1, 3, 4, 7, 11, 18, 29, 47, 76]
```

```
[2, 1, 3, 4, 7]
[2]
Input must be a positive integer.
[2, 1, 3, 4, 7, 11, 18, 29, 47, 76]
```

Task 20 (Vowel & Consonant Counter – Test Case Design)

- Function: Count vowels and consonants in string.
- Test Cases to Design:
- Normal: "hello" $\rightarrow (2,3)$

- Edge: "" $\rightarrow$ (0,0)
- Only vowels: "aeiou" $\rightarrow$ (5,0)

Large: Long text

- Requirement: Validate correctness with pytest.

Code:

```
#generate a code for vowels and consonants in string
#and provide test cases for the code and add pytest and test case to design
#normal function to count vowels and consonants in a string,edge case and only vowels and give length of the string
def count_vowels_consonants(input_string):
    """
    Count the number of vowels and consonants in a given string.

    Args:
        input_string (str): The string to be analyzed.
    Returns:
        tuple: A tuple containing the count of vowels and consonants.
    """
    vowels = 'aeiouAEIOU'
    vowel_count = 0
    consonant_count = 0
    for char in input_string:
        if char.isalpha():
            if char in vowels:
                vowel_count += 1
            else:
                consonant_count += 1
    return vowel_count, consonant_count

# Test cases
print(count_vowels_consonants("Hello")) # Output: (2, 3)
print(edge_case := count_vowels_consonants("")) # Output: (0, 0)
print(only_vowels := count_vowels_consonants("aeiou")) # Output: (5, 0)
```

Output:

```
(2, 3)
(0, 0)
(5, 0)
```